Plan d'action : Prédiction du prix des diamants avec le Machine Learning

Dataset : Diamonds Price Dataset – Kaggle

Objectif du projet

Réaliser un projet complet de Machine Learning supervisé permettant de prédire le prix d'un diamant à partir de ses caractéristiques. L'objectif est de comparer deux algorithmes de régression, d'évaluer leurs performances et de sélectionner le meilleur modèle en justifiant le choix à partir de résultats mesurables.

Étapes à réaliser

1. Exploration et analyse des données (EDA)

- Charger le dataset et examiner sa structure : `head()`, `shape()`, `info()` et `describe()`.
- Identifier les variables numériques et catégorielles.
- Analyser les valeurs manquantes, les doublons et les éventuelles incohérences.
- Étudier les distributions des variables à l'aide d'histogrammes et de boxplots.
- Analyser les relations entre les caractéristiques et le prix du diamant.
- Étudier les corrélations entre les variables numériques.
- Détecter les valeurs aberrantes (*outliers*) à l'aide de la méthode IQR et justifier leur traitement.

2. Nettoyage des données (Data Cleaning)

- Traiter les valeurs manquantes selon la nature des variables.
- Supprimer les doublons lorsque cela est justifié.
- Corriger les incohérences et les valeurs impossibles.
- Traiter les valeurs aberrantes en justifiant les décisions prises.
- Vérifier la qualité des données après nettoyage.

3. Préparation des données

- Définir la variable cible `y` correspondant au prix du diamant.
- Définir les variables explicatives `X`.
- Identifier les variables numériques, nominales et ordinales.
- Écarter les identifiants ou les variables non pertinentes, en justifiant leur exclusion.

4. Séparation des données (Train/Test Split)

- Séparer les données en un ensemble d'entraînement et un ensemble de test, par exemple avec une proportion de 80 % / 20 %.
- Utiliser `random_state` pour assurer la reproductibilité.
- Réaliser le split avant d'apprendre les paramètres des transformations afin d'éviter toute fuite de données (*Data Leakage*).

5. Prétraitement des données (Preprocessing)

- Standardiser les variables numériques lorsque cela est pertinent.
- Encoder les variables catégorielles nominales avec `OneHotEncoder`.
- Utiliser `OrdinalEncoder` pour les variables ordinales, si nécessaire, en définissant explicitement leur ordre.
- Regrouper les transformations dans un `ColumnTransformer`.
- Gérer les éventuelles valeurs manquantes à l'aide d'imputeurs adaptés.

6. Construction des pipelines

- Créer un pipeline complet intégrant le prétraitement et l'algorithme de régression.
- Construire un pipeline distinct pour chaque algorithme.
- Vérifier que les transformations sont apprises uniquement à partir des données d'entraînement de chaque fold.

7. Entraînement de deux algorithmes de régression

Choisir et entraîner les deux algorithmes suivants :

- **Algorithme 1 :** `LinearRegression` — modèle de référence permettant d'étudier les relations linéaires entre les caractéristiques et le prix.
- **Algorithme 2 :** `RandomForestRegressor` — modèle capable de capturer des relations non linéaires et des interactions entre les variables.

8. Validation croisée (Cross-Validation)

- Utiliser `KFold` ou la validation croisée intégrée à `GridSearchCV`.
- Appliquer une validation croisée à 5 folds, par exemple.
- Comparer les performances moyennes des deux algorithmes sur les mêmes données d'entraînement.
- Examiner la variabilité des scores entre les folds.

9. Optimisation des hyperparamètres avec `GridSearchCV`

- Appliquer `GridSearchCV` sur chaque pipeline.
- Définir une grille d'hyperparamètres pertinente pour chaque algorithme.
- Pour `LinearRegression`, tester les paramètres compatibles et utiles, ou conserver le modèle par défaut comme référence.
- Pour `RandomForestRegressor`, tester notamment `n_estimators`, `max_depth` et `min_samples_leaf`.
- Utiliser une métrique cohérente avec l'objectif, par exemple `neg_root_mean_squared_error`.
- Identifier les meilleurs hyperparamètres à l'aide de `best_params_` et `best_score_`.

10. Évaluation des modèles

Évaluer les meilleurs modèles sur le jeu de test, qui ne doit pas avoir été utilisé pendant l'entraînement ou l'optimisation.

Calculer et interpréter les métriques suivantes :

- **MAE** : erreur absolue moyenne, exprimée dans l'unité du prix.
- **MSE** : erreur quadratique moyenne, qui pénalise davantage les grandes erreurs.
- **RMSE** : racine de l'erreur quadratique moyenne, exprimée dans la même unité que le prix.

- **R^2** : mesure la part de variabilité du prix expliquée par le modèle.

11. Étude comparative et sélection du meilleur modèle

- Présenter les résultats des deux modèles dans un tableau comparatif.
- Comparer les scores de validation croisée et les métriques obtenues sur le jeu de test.
- Examiner les écarts entre les performances d'entraînement et de test afin de détecter un éventuel surapprentissage.
- Sélectionner le meilleur modèle en fonction des résultats, de sa capacité de généralisation et de l'objectif métier.
- Justifier clairement le choix final.

12. Conclusion et livrables

- Présenter les principales observations issues de l'EDA.
- Résumer les traitements appliqués aux données.
- Expliquer les choix de prétraitement et les hyperparamètres retenus.
- Présenter l'étude comparative et justifier le modèle sélectionné.
- Organiser le code en sections claires et reproductibles.
- Fournir un notebook contenant les étapes, les visualisations, les résultats et les interprétations.
- Sauvegarder le meilleur pipeline entraîné avec `joblib` afin de pouvoir le réutiliser pour effectuer de nouvelles prédictions.

Critères de réussite

- Respect de l'ordre des étapes.
- Absence de fuite de données.
- Utilisation correcte des pipelines et de la validation croisée.
- Optimisation des hyperparamètres.
- Interprétation des métriques.
- Justification du modèle final.

Consigne importante

Le nettoyage exploratoire peut être réalisé avant le split, mais les paramètres appris à partir des données — imputation, standardisation et encodage — doivent être intégrés au pipeline et ajustés uniquement sur les données d'entraînement. Le jeu de test doit rester indépendant jusqu'à l'évaluation finale.